HỌC VIỆN CÔNG NGHỆ BƯU CHÍNH VIỄN THÔNG

KHOA ĐÀO TẠO SAU ĐẠI HỌC

BÀI TẬP CUỐI KỲ

MÔN: NHẬN DẠNG MẪU

Tên đề tài: Nhận dạng ký tự chữ viết tay

Nhóm thực hiện: Nhóm 27

Lớp hành chính: M26CQKH02-B

Giảng viên hướng dẫn: TS. Nguyễn Xuân Đức

Hà Nội, tháng 10 năm 2026

MỤC LỤC

DANH MỤC HÌNH 3
DANH MỤC BẢNG 3
MỞ ĐẦU 4
CHƯƠNG 1: GIỚI THIỆU 5
1.1. Đặt vấn đề 5
1.2. Mục tiêu 5
CHƯƠNG 2: GIỚI THIỆU BỘ DỮ LIỆU EMNIST BYMERGE 6
2.1. Tổng quan về EMNIST 6
2.2. Thống kê dữ liệu 7
2.3. Trực quan hóa dữ liệu 7
CHƯƠNG 3: PHÂN TÍCH DỮ LIỆU 8
3.1. Phân bố nhãn 8
3.2. Đặc trưng ảnh và thách thức 8
3.3. Tiền xử lý dữ liệu 9
CHƯƠNG 4: PHƯƠNG PHÁP 10
4.1. Mạng Perceptron đa lớp (MLP) 10
4.2. Mạng nơ-ron tích chập (CNN) 11
4.3. Siêu tham số và kỹ thuật điều chuẩn 13
4.4. Cấu hình huấn luyện 16
CHƯƠNG 5: THỰC NGHIỆM VÀ KẾT QUẢ 17
5.1. Kết quả huấn luyện MLP 17
5.2. Kết quả huấn luyện CNN 20
5.3. So sánh hai mô hình 23
CHƯƠNG 6: PHÂN TÍCH LỖI 24
6.1. Phân tích ma trận nhầm lẫn 24
6.2. Các cặp nhầm lẫn phổ biến 24
6.3. Nguyên nhân và hướng cải thiện 24
CHƯƠNG 7: KẾT LUẬN VÀ HẠN CHẾ 26
7.1. Kết luận 26
7.2. Hạn chế 26
7.3. Hướng phát triển 27
TÀI LIỆU THAM KHẢO 28

DANH SÁCH NHÂN SỰ NHÓM 27

Họ và tên	MHV
Nguyễn Hoàng Nam	B26CHKH074
Phan Đỗ Hải Minh	B26CHKH071

DANH MỤC HÌNH

Hình 1: Minh họa 47 nhãn của bộ dữ liệu EMNIST ByMerge

Hình 2: Một số mẫu dữ liệu từ EMNIST ByMerge

Hình 3: Kiến trúc mạng Perceptron đa lớp (MLP)

Hình 4: Minh họa mạng nơ-ron tích chập (CNN)

Hình 5: Đồ thị độ chính xác của MLP qua các epoch

Hình 6: Đồ thị hàm mất mát của MLP qua các epoch

Hình 7: Ma trận nhầm lẫn của MLP

Hình 8: Một số mẫu dự đoán của MLP

Hình 9: Một số mẫu dự đoán sai của MLP

Hình 10: Đồ thị độ chính xác của CNN qua các epoch

Hình 11: Đồ thị hàm mất mát của CNN qua các epoch

Hình 12: Ma trận nhầm lẫn của CNN

Hình 13: Một số mẫu dự đoán của CNN

Hình 14: Một số mẫu dự đoán sai của CNN

DANH MỤC BẢNG

Bảng 1: Thống kê số lượng mẫu EMNIST ByMerge

Bảng 2: Siêu tham số của mô hình MLP

Bảng 3: Siêu tham số của mô hình CNN

Bảng 4: So sánh kết quả giữa MLP và CNN

MỞ ĐẦU

Báo cáo này trình bày quá trình xây dựng và đánh giá hai mô hình học máy mạng nơ-ron nhân tạo: Multi-layer Perceptron (MLP) và Convolutional Neural Network (CNN) cho bài toán nhận dạng ký tự viết tay trên bộ dữ liệu EMNIST ByMerge với 47 lớp:

- **Chữ số (10 lớp):** Từ 0 đến 9.
- **Chữ cái viết hoa (26 lớp):** Từ A đến Z.
- **Chữ cái viết thường (11 lớp):** Chỉ bao gồm các chữ cái có dạng viết thường khác biệt rõ rệt so với chữ viết hoa của chúng, bao gồm: a, b, d, e, f, g, n, q, r, t.

Bộ dữ liệu có tổng cộng 814.255 mẫu, trong đó 697.931 mẫu dùng để huấn luyện và 116.324 mẫu dùng để kiểm thử.

Cả hai mô hình đều được huấn luyện với cùng một pipeline: tải dữ liệu từ CSV, xáo trộn, chuẩn hóa về khoảng [0, 1], mã hóa one-hot nhãn, huấn luyện 10 epoch với batch size 256 bằng thuật toán Adam và hàm mất mát categorical cross-entropy. Mô hình MLP đạt độ chính xác trên 88% trên tập kiểm thử, trong khi mô hình CNN đạt khoảng 90%. Kết quả cho thấy CNN vượt trội hơn MLP trong bài toán nhận dạng ảnh nhờ khả năng trích xuất đặc trưng không gian thông qua các lớp tích chập.

Đường dẫn source code: https://github.com/LuminNguyen0402/Character-recognition-PTIT.git

CHƯƠNG 1: GIỚI THIỆU

1.1. Đặt vấn đề

Nhận dạng ảnh (Image Recognition) là một lĩnh vực con của Thị giác máy tính (Computer Vision) và Trí tuệ nhân tạo, đã mang lại nhiều giải pháp cho các bài toán thực tiễn trong cuộc sống. Quá trình này có thể được áp dụng trong nhiều lĩnh vực khác nhau như y tế (chẩn đoán bệnh qua hình ảnh), giao thông (phát hiện vi phạm), an ninh (phân biệt con người), tài chính (đọc hóa đơn, chứng từ),.. Có thể khẳng định rằng nhận dạng ảnh là một trong những chủ đề nóng của ngành Khoa học máy tính.

Theo yêu cầu của môn học Nhận dạng mẫu, bài tập cuối kỳ yêu cầu sinh viên tìm hiểu và thực hiện một bài toàn nhận dạng mẫu trên dữ liệu thực tế. Để đạt được mục tiêu này, nhóm tập trung nghiên cứu hai mô hình nhận dạng ảnh phổ biến trên bộ dữ liệu EMNIST: Multi-layer Perceptron (MLP) và Convolutional Neutral Network (CNN). Đây là hai mô hình nền tảng, đã được chứng minh có hiệu năng tốt trong bài toán nhận dạng ảnh. Nhóm triển khai cả hai mô hình để có cái nhìn tổng quan về cấu trúc cũng như tiếp cận nhiều thuật ngữ liên quan đến học máy.

1.2. Mục tiêu

Mục tiêu chính của bài tập cuối kỳ này bao gồm:

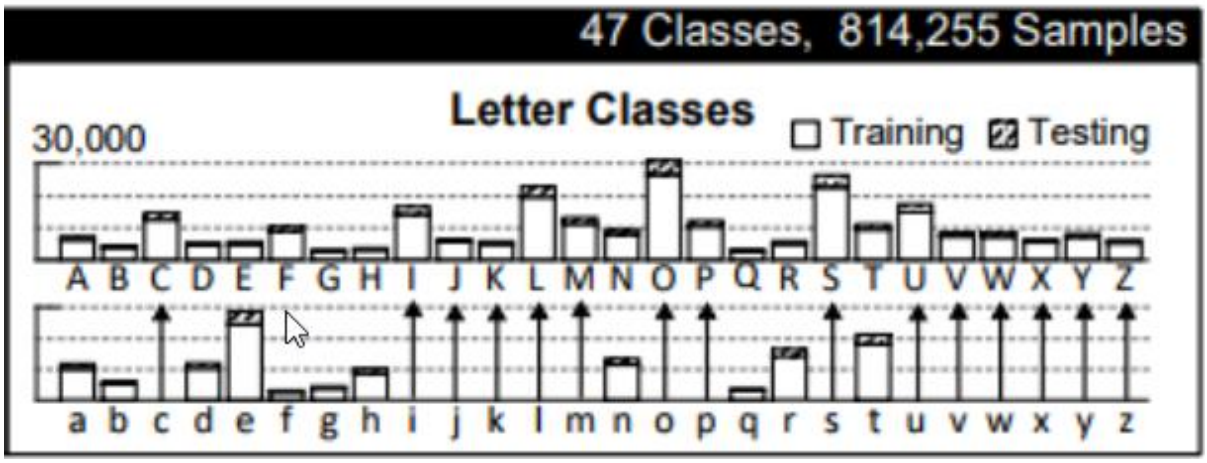
(1) Tìm hiểu và nắm vững lý thuyết về mạng nơ-ron nhân tạo, đặc biệt là hai kiến trúc MLP và CNN.
(2) Xây dựng và huấn luyện thành công hai mô hình trên bộ dữ liệu EMNIST By Merge với 47 lớp.
(3) So sánh hiệu năng giữa hai mô hình về độ chính xác, tốc độ huấn luyện và khả năng tổng quát hóa.
(4) Phân tích lỗi dự đoán, xác định các cặp ký tự dễ bị nhầm lẫn và đề xuất hướng cải thiện.
(5) Rút ra bài học kinh nghiệm về vai trò của tiền xử lý dữ liệu, siêu tham số và kỹ thuật điều chỉnh trong huấn luyện mạng nơ-ron.

CHƯƠNG 2: GIỚI THIỆU BỘ DỮ LIỆU EMNIST BYMERGE

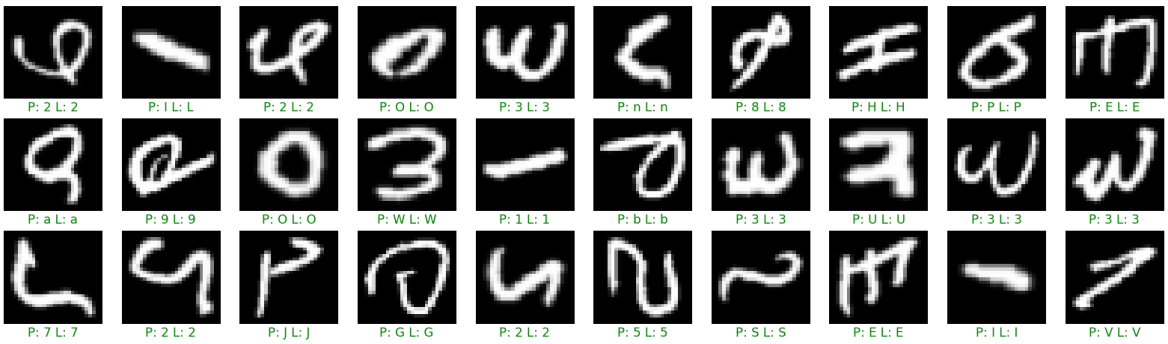
2.1. Tổng quan về EMNIST

Bộ dữ liệu EMNIST (Extended MNIST) là tập dữ liệu các ký tự viết tay được tạo ra từ cơ sở dữ liệu NIST Special Database 19, chứa các ký tự viết tay gồm: chữ số, chữ cái viết hoa và viết thường. Tập dữ liệu này được chuyển đổi sang định dạng ảnh 28x28 pixel và cấu trúc tương thích với bộ dữ liệu MNIST nổi tiếng. So với MNIST, EMNIST khó hơn cho các mô hình nhận dạng do sự đa dạng của dữ liệu như xoay, lật, mờ,… Tuy nhiên, điều này cũng giúp tăng khả năng nhận dạng các kiểu viết không theo dạng chuẩn cũng như cải thiện độ chính xác.

Trong bài tập này, nhóm sử dụng phiên bản ByMerge của EMNIST từ Kaggle, bao gồm 814.255 mẫu ký tự viết tay (697.931 mẫu huấn luyện và 116.324 mẫu kiểm thử). Dữ liệu được lưu trữ dưới dạng file CSV, trong đó cột đầu tiên là nhãn của ảnh, các cột còn lại là giá trị pixel của ảnh 28x28 (tức 784 cột), mỗi hàng là một mẫu ảnh. Tổng cộng có 47 nhãn, bao gồm 10 chữ số từ 0 đến 9 và 37 chữ cái (các chữ cái có cùng hình dạng giữa viết hoa và viết thường sẽ được gộp lại thành một nhãn).



Hình 1: Minh họa 47 nhãn của bộ dữ liệu EMNIST ByMerge



Hình 2: Một số mẫu dữ liệu từ EMNIST ByMerge

2.2. Thống kê dữ liệu

Thuộc tính	Giá trị
Tổng số mẫu	814.255
Số mẫu huấn luyện	697.931
Số mẫu kiểm thử	116.324
Số lớp (nhãn)	47
Kích thước ảnh	28 x 28 pixel (784 đặc trưng)

Bảng 1: Thống kê số lượng mẫu trong EMNIST ByMerge

Đặc trưng của mỗi mẫu dữ liệu:

- Nhãn (label): một số nguyên trong khoảng [0, 46] đại diện cho lớp của ký tự.
- Pixel: 784 giá trị số nguyên trong khoảng [0, 255] biểu thị cường độ xám của từng pixel trong ảnh 28x28.
- Định dạng lưu trữ: file CSV với cột đầu tiên là nhãn, 784 cột tiếp theo là giá trị pixel.

2.3. Trực quan hóa dữ liệu

Để hiểu rõ hơn về đặc điểm của dữ liệu, nhóm tiến hành trực quan hóa một số mẫu ngẫu nhiên từ tập huấn luyện. Mỗi ảnh có kích thước 28x28 pixel, hiển thị dưới dạng ảnh xám. Quan sát cho thấy:

- Các ký tự có nhiều kiểu viết khác nhau, phản ánh sự đa dạng của người viết.
- Một số ký tự có hình dạng rất giống nhau (ví dụ: chữ O và số 0, chữ I và số 1, chữ l và số 1), gây khó khăn cho mô hình phân loại.
- Ảnh trong EMNIST được lưu trữ ở dạng đã xoay và lật (transpose), tức chiều của ảnh bị đảo ngược so với cách hiển thị thông thường. Đây là một đặc điểm quan trọng cần lưu ý trong quá trình tiền xử lý.

CHƯƠNG 3: PHÂN TÍCH DỮ LIỆU

3.1. Phân bố nhãn

Bộ dữ liệu EMNIST ByMerge có 47 lớp, trong đó 10 lớp đầu tiên (0-9) là các chữ số, các lớp còn lại (10-46) là các chữ cái. Phân bố số lượng mẫu giữa các lớp không đồng đều hoàn toàn - một số lớp có nhiều mẫu hơn các lớp khác do tần suất xuất hiện của các ký tự trong ngôn ngữ tự nhiên và cách gộp nhóm trong phiên bản ByMerge.

Sự mất cân bằng nhẹ giữa các lớp này có thể ảnh hưởng đến hiệu năng của mô hình, đặc biệt với các lớp có ít mẫu. Tuy nhiên, với tổng số mẫu lớn (gần 700.000 mẫu huấn luyện), ảnh hưởng này được giảm thiểu đáng kể.

3.2. Đặc trưng ảnh và thách thức

Qua qua trinh kham pha du lieu, nhom nhan thay mot so dac diem va thach thuc quan trong:

(1) Ảnh được lưu trữ ở dạng transpose: Trong file CSV, ảnh được lưu theo cách đã xoay 90 độ và lật ngang so với hướng hiển thị tự nhiên. Điều này có nghĩa là khi đọc ảnh từ CSV và hiển thị trực tiếp, ký tự sẽ bị xoay ngược. Trong bài tập này, nhóm KHÔNG thực hiện thao tác np.transpose/np.flip để đưa ảnh về đúng hướng, đây là một trong những nguyên nhân gây ra lỗi nhận dạng giữa các cặp ký tự có hình dạng tương tự..

(2) Các cặp ký tự dễ nhầm lẫn: Do đặc điểm của bộ dữ liệu và cách viết tay, một số cặp ký tự có hình dạng rất giống nhau, bao gồm:
- Chữ O va số 0
- Chữ I, chữ l va số 1
- Chữ Z va số 2
- Chữ S va số 5
- Chữ G va số 6
- Chữ B va số 8

(3) Đa dạng kiểu viết: Mỗi ký tự có thể được viết theo nhiều kiểu khác nhau (in hoa, in thường, nghiêng, đứng,...), làm tăng độ phức tạp của bài toán.

(4) Nhiễu trong ảnh: Một số ảnh có thể chứa nhiễu, nét viết không rõ ràng hoặc bị cắt xén, ảnh hưởng đến chất lượng nhận dạng.

3.3. Tiền xử lý dữ liệu

Quy trình tiền xử lý dữ liệu được áp dụng thống nhất cho cả hai mô hình:

Bước 1 - Tải dữ liệu: Sử dụng thư viện pandas để đọc file CSV. Cột đầu tiên được tách thành nhãn (y), 784 cột còn lại tạo thành vector đặc trưng (X).

Bước 2 - Xáo trộn: Dữ liệu được xáo trộn ngẫu nhiên để đảm bảo phân bố đồng đều giữa các batch trong quá trình huấn luyện.

Bước 3 - Chuẩn hóa: Giá trị pixel được chia cho 255 để đưa về khoảng [0, 1]. Việc này giúp ổn định quá trình huấn luyện và tăng tốc độ hội tụ.

Bước 4 - Mã hóa nhãn: Nhãn dạng số nguyên được chuyển sang dạng one-hot vector có độ dài 47 bằng hàm to_categorical của Keras.

> Ví dụ: nhãn 5 -> vector [0,0,0,0,1,0,...,0].

Bước 5 - Định dạng lại cho CNN: Đối với mô hình CNN, dữ liệu được reshape từ vector 784 chiều về tensor 3 chiều [28, 28, 1] để phù hợp với đầu vào của lớp tích chập. Mô hình MLP giữ nguyên định dạng vector phẳng 784 chiều.

Bước 6 - Ép kiểu float16: Để tiết kiệm bộ nhớ, dữ liệu được ép kiểu về float16.

> Lưu ý: các cell trực quan hóa sau đó nhân ngược lại với 255 và ép kiểu uint8 để hiển thị bằng imshow.

CHƯƠNG 4: PHƯƠNG PHÁP

4.1. Mạng Perceptron đa lớp (MLP)

4.1.1. Khái niệm mạng nơ-ron nhân tạo

Khái niệm mạng nơ-ron nhân tạo (Artificial Neural Network - ANN) được lấy cảm hứng từ hệ thần kinh sinh học, trong đó mạng lưới nơ-ron đóng vai trò quan trọng trong cơ thể con người. Toàn bộ quá trình nhận và truyền tín hiệu được thực hiện theo một cách thức đặc biệt: một nơ-ron nhận tín hiệu từ các nơ-ron khác thông qua các đuôi gai (dendrite), sau đó gửi tín hiệu qua sợi trục (axon) thông qua các khớp thần kinh (synapse) đến các nơ-ron khác.

Tương tự như nơ-ron sinh học, một nơ-ron nhân tạo có nhiều kênh đầu vào, một giai đoạn xử lý và một đầu ra có thể phân nhánh đến nhiều nơ-ron nhân tạo khác. Giai đoạn xử lý có thể bao gồm nhiều phần tử xử lý đơn giản được kết nối với nhau, nhận các đối số như trọng số, đầu vào,... và tính toán đầu ra bằng cách truyền chúng qua hàm kích hoạt.

Bằng cách xếp chồng nhiều nơ-ron nhân tạo, ta tạo thành một lớp (layer), ghép các lớp lại với nhau sẽ tạo thành một mạng nơ-ron đa lớp. MLP là ví dụ cơ bản nhất của mạng nơ-ron đa lớp.

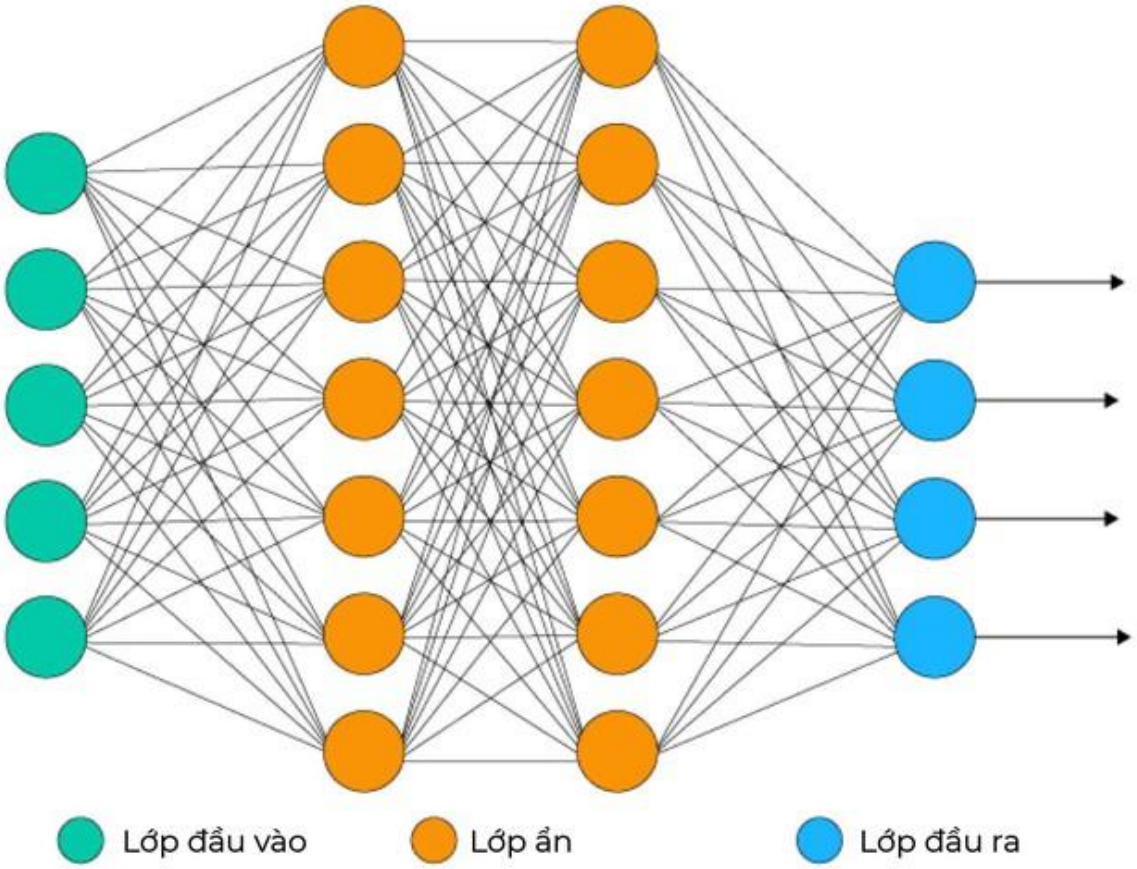
4.1.2. Kiến trúc MLP trong bài tập

Mô hình MLP được xây dựng với kiến trúc sau:

- Lớp đầu vào (Input layer): nhận vector 784 chiều (ảnh 28x28 được làm phẳng).
- Lớp ẩn 1 (Hidden layer 1): Dense(512) với hàm kích hoạt ReLU.
- Lớp Dropout 1: Dropout(0.2) - loại bỏ ngẫu nhiên 20% nơ-ron để giảm overfitting.
- Lớp ẩn 2 (Hidden layer 2): Dense(512) với hàm kích hoạt ReLU."
- Lớp Dropout 2: Dropout(0.2).
- Lớp đầu ra (Output layer): Dense(47) với hàm kích hoạt softmax,cho ra phân phối xác suất trên 47 lớp.

Khởi tạo trọng số: Theo mặc định của Keras:

- Trọng số (Kernel): Glorot Uniform, lấy mẫu trong khoảng [-limit (input + output)).
- Độ chệch (Biases): khởi tạo bằng 0.



Hình 3: Kiến trúc mạng Perceptron đa lớp (MLP)

4.2. Mạng nơ-ron tích chập (CNN)

4.2.1. Giới thiệu về CNN

CNN được biết đến là một trong những mô hình nhận dạng ảnh tốt nhất hiện nay. Vấn đề của các mô hình mạng nơ-ron kết nối đầy đủ (fully connected) là khi xử lý ảnh có kích thước lớn, số lượng pixel cũng rất lớn, lớp đầu vào có thể có n^2 đơn vị (với ảnh vuông), cộng với trọng số và các lớp ẩn, số tham số tăng nhanh. Phép tích chập giúp giải quyết vấn đề này đồng thời vẫn trích xuất được các đặc trưng của ảnh.

Ngoài các lớp kết nối đầy đủ cơ bản, CNN có thêm các lớp tích chập trước khi xử lý. Cấu trúc cơ bản:

Ảnh đầu vào -> Lớp tích chập (Conv) + Lớp gộp (Pool) -> Lớp kết nối đầy đủ (FC) -> Đầu ra

Có nhiều kiến trúc CNN nổi tiếng như LeNet, AlexNet, GoogleNet, VGG Net, ResNet [27]. Trong bài tập này, nhóm sử dụng kiến trúc CNN truyền thống.

4.2.2. Kiến trúc CNN trong bài tập

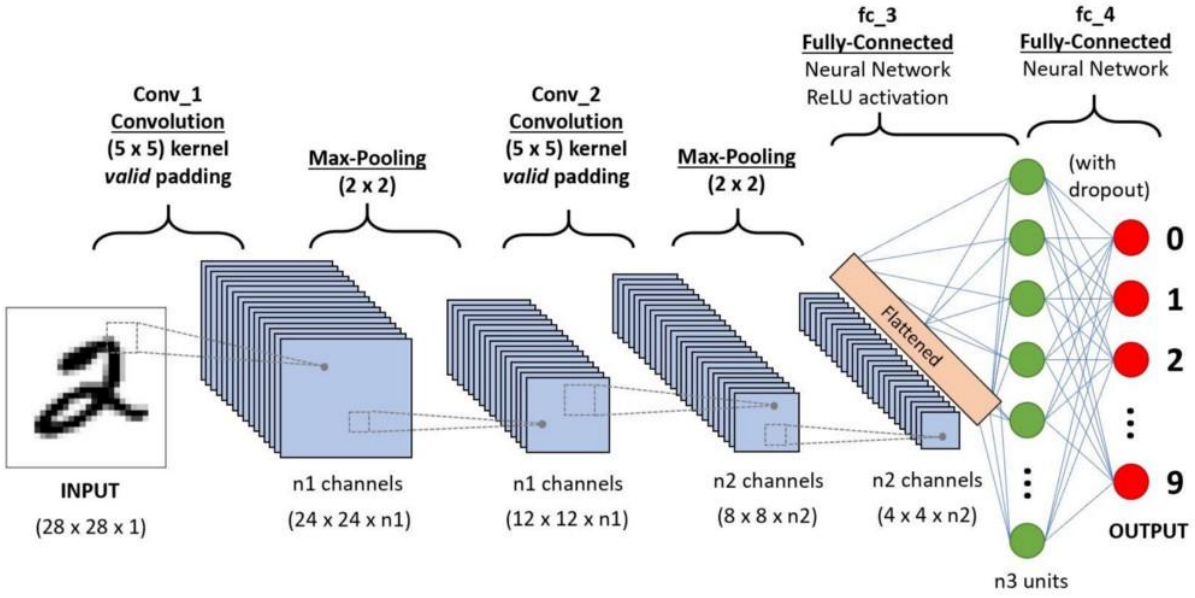
Mô hình CNN được xây dựng với 2 khối tích chập (Convolutional Block), mỗi khối gồm:

- 2 lớp Conv2D với hàm kích hoạt LeakyReLU (alpha = 0.3 mặc định).
- 1 lớp MaxPool2D để giảm kích thước không gian.
- 1 lớp Dropout để giảm overfitting.

Khối 1: Conv2D(32, 3, padding=same) -> LeakyReLU -> Conv2D(32, 3, padding=same) -> LeakyReLU -> MaxPool2D(2,2) -> Dropout(0.25)

Khối 2: Tương tự khối 1 nhưng với 64 filters.

Phần đầu phân loại (Head): Flatten -> Dense(512, ReLU) -> Dense(128, ReLU) -> Dense(47, softmax)



Hình 4: Minh họa mạng nơ-ron tích chập (CNN)

4.2.3. Quy trình hoạt động

Bước 1 - Tích chập: Bộ lọc (filter) kích thước 3x3 được trượt trên ma trận ảnh và nhân từng phần với ảnh. Kết quả là ma trận tích chập 28x28 (giữ nguyên kích thước nhờ padding=same). Với 32 bộ lọc, ta có 32 ma trận khác nhau, ảnh đầu ra có shape [28, 28, 32].

Bước 2 - LeakyReLU: Áp dụng hàm kích hoạt LeakyReLU lên kết quả tích chập.

Bước 3 - Max Pooling: Sử dụng cửa sổ 2x2 để giảm kích thước ảnh từ 28x28 xuống 14x14 bằng cách lấy giá trị lớn nhất trong mỗi vùng 2x2.

Bước 4 - Lặp lại: Quy trình tương tự được lặp lại với 64 filters ở khối thứ hai, giảm kích thước xuống 7x7.

Bước 5 - Flatten và Dense: Dữ liệu được làm phẳng thành vector và đưa qua các lớp Dense để phân loại.

4.3. Siêu tham số và kỹ thuật điều chuẩn

4.3.1. Dropout

Trong mạng nơ-ron, kỹ thuật điều chuẩn được sử dụng để giảm overfitting bằng cách ngăn chặn sự đồng thích nghi (co-adaptation) trên dữ liệu huấn luyện là dropout. Trong quá trình huấn luyện, kỹ thuật dropout được sử dụng để loại bỏ ngẫu nhiên các nơ-ron. Sau mỗi lớp Dense trong MLP hoặc sau mỗi khối tích chập trong CNN, dropout được áp dụng để giảm vấn đề overfitting.

Khi sử dụng mạng nơ-ron kết nối đầy đủ, các nơ-ron phụ thuộc lẫn nhau. Kỹ thuật này buộc mô hình phải tìm các đặc trưng mạnh mẽ hơn với xác suất p loại bỏ nơ-ron tốt để huấn luyện các nơ-ron khác. Trong hai mô hình, nhóm chọn p trong khoảng 0.2 - 0.25, tức 20% đến 25% số nơ-ron sẽ bị loại bỏ.

4.3.2. Batch size, số epoch và learning rate

Batch size là siêu tham số xác định số lượng mẫu được xử lý trước khi cập nhật tham số mô hình. Nếu chia dữ liệu thành nhiều batch, ta có thể tiết kiệm bộ nhớ. CNN đặc biệt nhạy cảm với batch size. Trong bài tập này, nhóm chọn batch size = 256.

Số epoch là số lần truyền hoàn toàn qua tập huấn luyện. Nhóm chọn epochs = 10 cho mỗi mô hình. Lý do chọn giá trị nhỏ này là vì tập dữ liệu lớn nên cần ít epoch hơn để đạt độ chính xác tốt nhất. Nếu số epoch quá lớn, mô hình có thể bị overfitting.

Learning rate là tham số điều chỉnh trong thuật toán tối ưu, xác định kích thước bước tại mỗi vòng lặp khi tiến tới cực tiểu của hàm mất mát. Để chọn learning rate tốt nhất cần thử nhiều giá trị. Trong bài tập này, nhóm sử dụng giá trị mặc định do hạn chế về thời gian và phần cứng.

4.3.3. Hàm kích hoạt

ReLU (Rectified Linear Unit): Là hàm kích hoạt phổ biến nhất trong mạng nơ-ron hiện đạt, được định nghĩa như sau:

f(x) = max(0, x)

ReLU giúp giải quyết vấn đề vanishing gradient và tăng tốc độ hội tụ so với sigmoid/tanh.

Softmax: Chuẩn hóa đầu ra của mạng thành phân phối xác suất trên các lớp dự đoán:

$$\sigma(\mathbf{z})_i = \frac{e^{z_i}}{\sum_{j=1}^{K} e^{z_j}}$$

Leaky ReLU: Biến thể của ReLU giải quyết vấn đề dead neuron (nơ-ron chết) khi ReLU luôn có giá trị âm:

$$y_i = \begin{cases} x_i \text{ if } x_i \geq 0 \\ \frac{x_i}{a_i} \text{ if } x_i < 0 \end{cases}$$

4.3.4. Bộ tối ưu Stochastic Grandient Descent và Adam

Stochastic Grandient Descent (SGD): Là thuật toán cơ bản giúp tối ưu hóa mạng nơ-ron, làm cho mạng nơ-ron hội tụ. Thuật toán Stochastic Gradient Descent chỉ tính toán chi phí của một ví dụ trong mỗi bước. Công thức SGD được sử dụng để cập nhật tham số trong quá trình truyền ngược, cụ thể là trọng số của mỗi lớp, sử dụng thuật toán lan truyền ngược (được sử dụng trong hầu hết các trình tối ưu hóa để cập nhật trọng số) để tính toán độ dốc:

$$\theta = \theta - \eta * \nabla_\theta J\ (\ \theta\ ;x\ ,y\ )$$

- θ là trọng số của mạng nơ-ron. Chúng ta cần cập nhật trọng số của nó để có được kết quả chính xác và như mong muốn.
- Tốc độ học tập η
- *J là t*huật ngữ chính thức là hàm mục tiêu, nhưng thường được gọi là hàm chi phí hoặc hàm tổn thất.

Ưu điểm và nhược điểm của Stochastic Grandient Descent:

- Ưu điểm:
    - Nhanh hơn so với phương pháp Gradient Descent thông thường.
- Nhược điểm:
    - So với các thuật toán mới hơn, thuật toán này hội tụ chậm hơn.
    - Nó gặp nhiều vấn đề hơn về việc bị kẹt ở điểm cực tiểu cục bộ so với các phương pháp mới hơn.

Adam (Adaptive Moment Estimation) là phương pháp có learning rate thích ứng, nghĩa là nó tính toán learning rate riêng cho từng tham số. Adam kết hợp ưu điểm của hai extension của SGD:

- SGD với Momentum (AdaGrad): Thuật toán Gradient Descent ngẫu nhiên có tích lũy động lượng
- Root Mean Square Propagation (RMSProp): Phương pháp lan truyền bình phương trung bình gốc

Công thức cập nhật trọng số:

$$w_t = w_{t-1} - \eta \frac{N}{\hat{v} + \epsilon}$$

Trong đó:

$$\hat{m} = \frac{m_t}{1 - \beta_1^t}$$

$$\hat{v} = \frac{v_t}{1 - \beta_2^t}$$

$$m_t = \beta_1 m_{t-1} + (1 - \beta_1) g_t$$

$$v_t = \beta_2 v_{t-1} + (1 - \beta_2) g_t$$

Từ đó nhóm chọn bộ tối ưu Adam: hội tụ nhanh hơn, kết quả ổn định và độ chính xác vượt trội so với SGD trong thực nghiệm.

4.4. Cấu hình huấn luyện

Siêu tham số	Giá trị
Optimizer	Adam
Learning rate	0.001 (mac dinh)
Batch size	256
Epochs	10
Validation split	0.1
Loss function	Categorical Cross-entropy
Metrics	Accuracy

Bảng 2: Siêu tham số của mô hình MLP

Siêu tham số	Giá trị
Optimizer	Adam
Learning rate	0.001 (mac dinh)
Batch size	256
Epochs	10
Validation split	0.1
Loss function	Categorical Cross-entropy
Metrics	Accuracy

Bảng 3: Siêu tham số của mô hình CNN

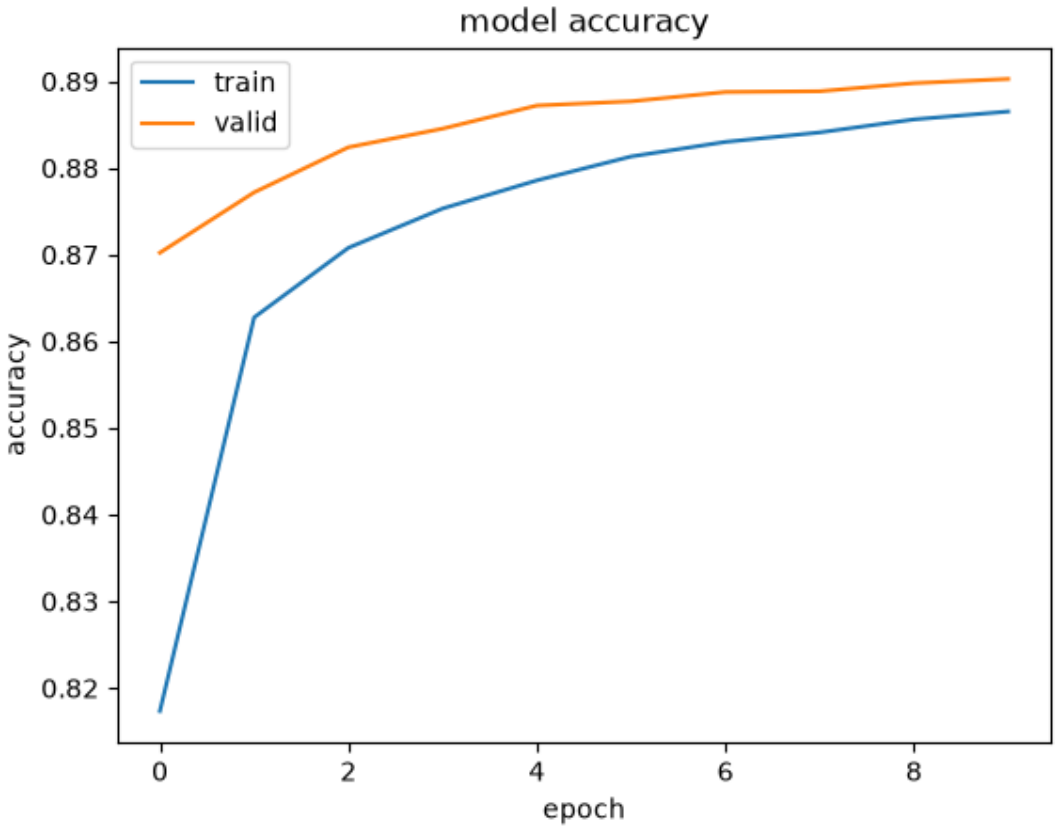
CHƯƠNG 5: THỰC NGHIỆM VÀ KẾT QUẢ

Trong chương này, nhóm trình bày kết quả huấn luyện và kiểm thử của hai mô hình MLP và CNN trên bộ dữ liệu EMNIST ByMerge. Quy trình thực nghiệm bao gồm:

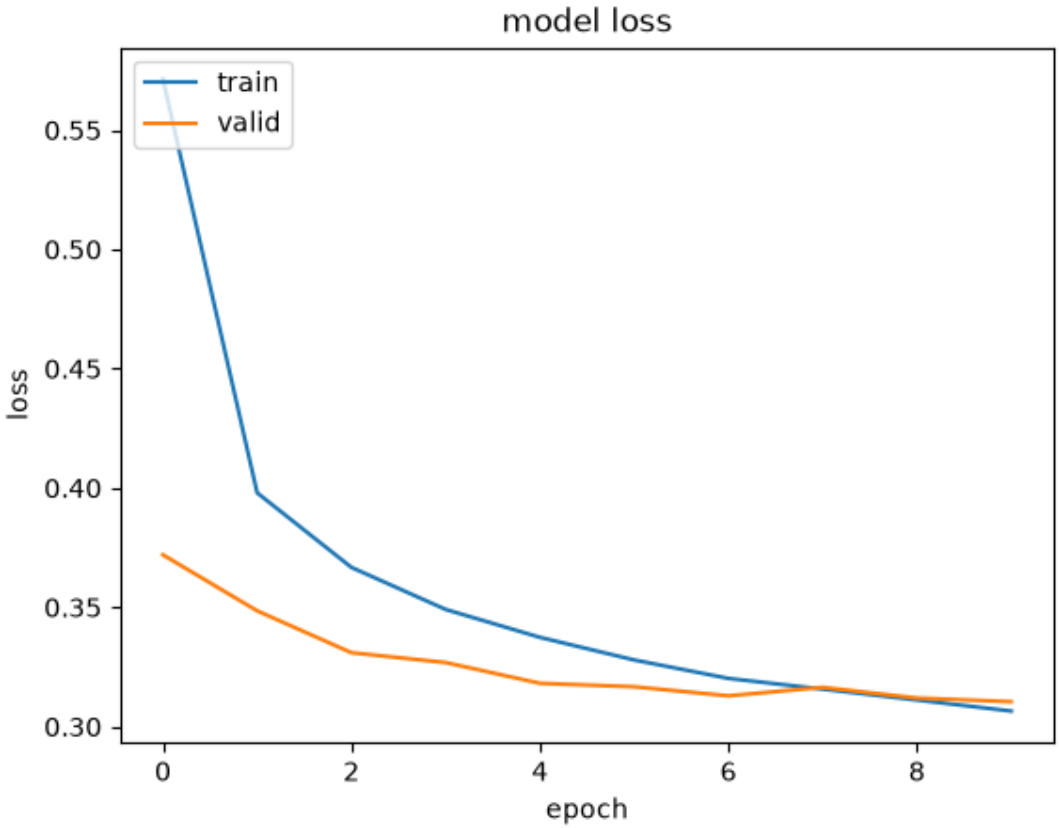
- Chia tập huấn luyện thành 90% huấn luyện và 10% validation (validation_split=0.1).
- Vẽ đồ thị accuracy và loss trong quá trình huấn luyện để phát hiện overfitting.
- Dự đoán trên tập test, sinh ma trận nhầm lẫn, hiển thị một số mẫu dự đoán đúng và sai.

5.1. Kết quả huấn luyện MLP

Dưới đây là các đồ thị thể hiện độ chính xác và hàm mất mát của mô hình MLP qua 10 epoch:

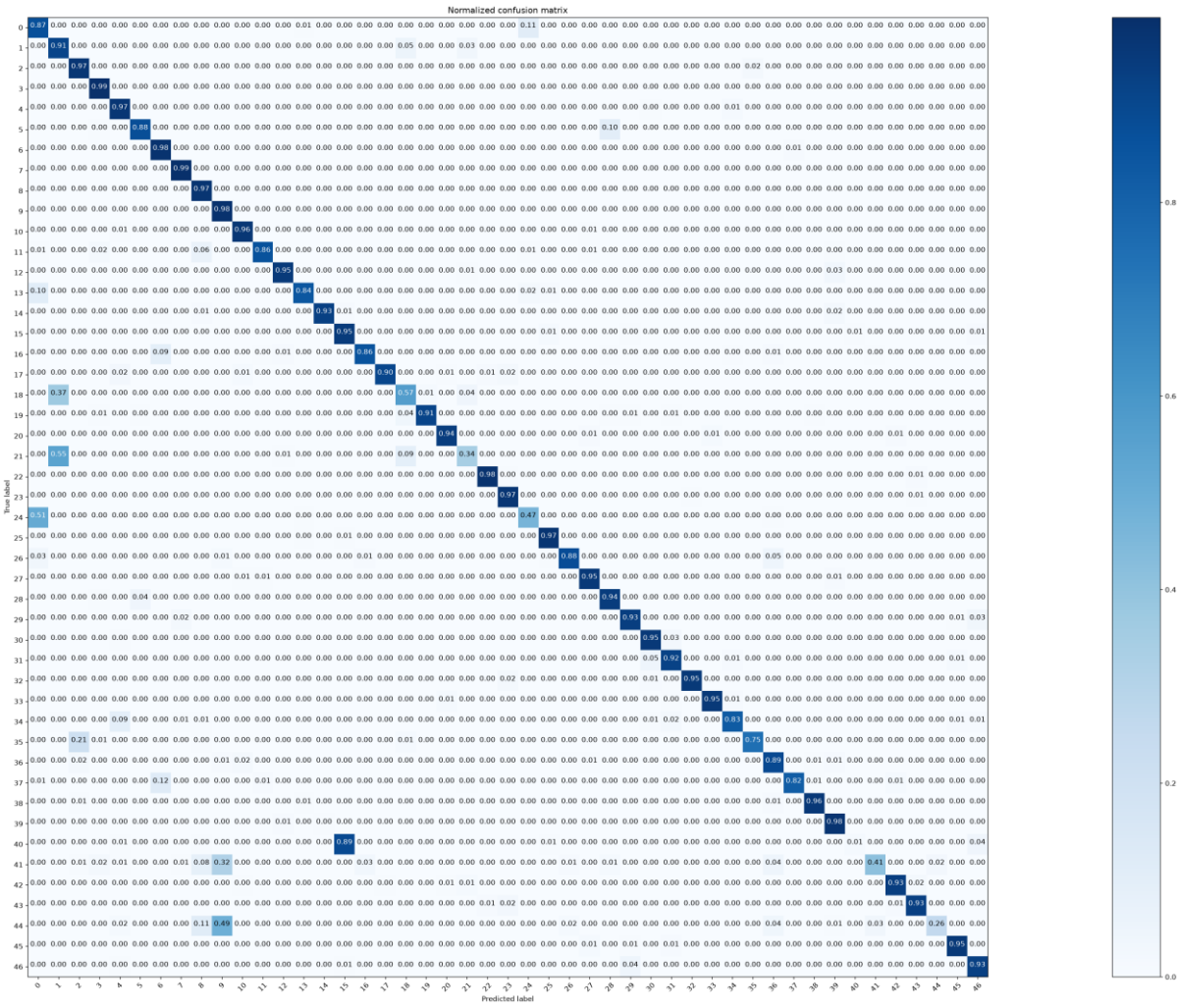


Hình 5: Đồ thị độ chính xác của MLP qua các epoch

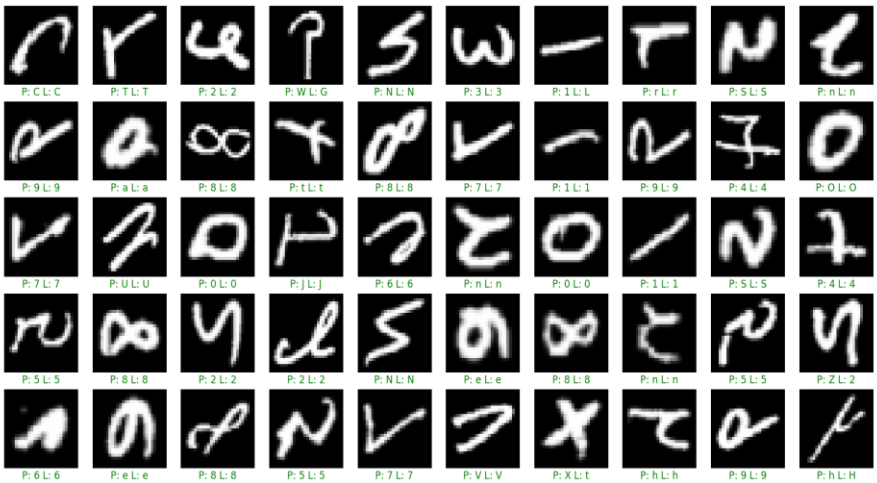


Hình 6: Đồ thị hàm mất mát của MLP qua các epoch

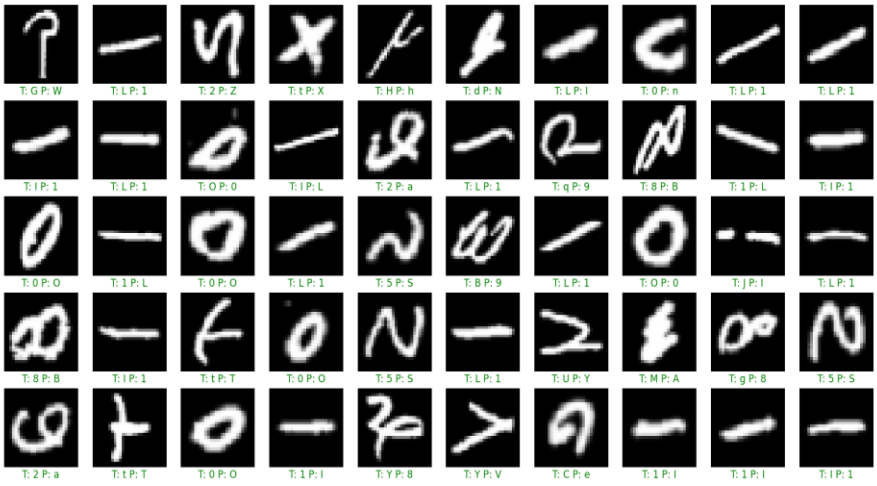
Quan sát: Cả đường accuracy huấn luyện và validation đều tăng đều qua các epoch, đường validation bám sát đường huấn luyện, đạt khoảng 88% sau 10 epoch. Hàm mất mát giảm đều, không có dấu hiệu overfitting rõ rệt.



Hình 7: Ma trận nhầm lẫn của MLP



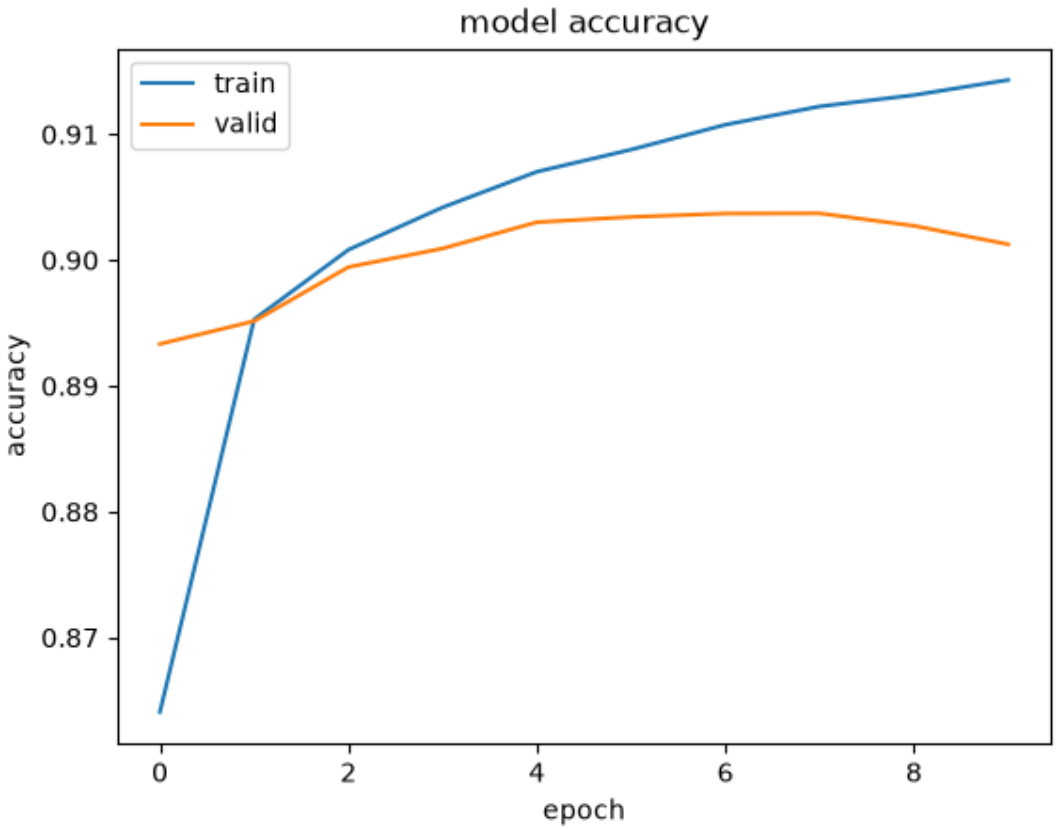
Hình 8: Một số mẫu dự đoán của MLP



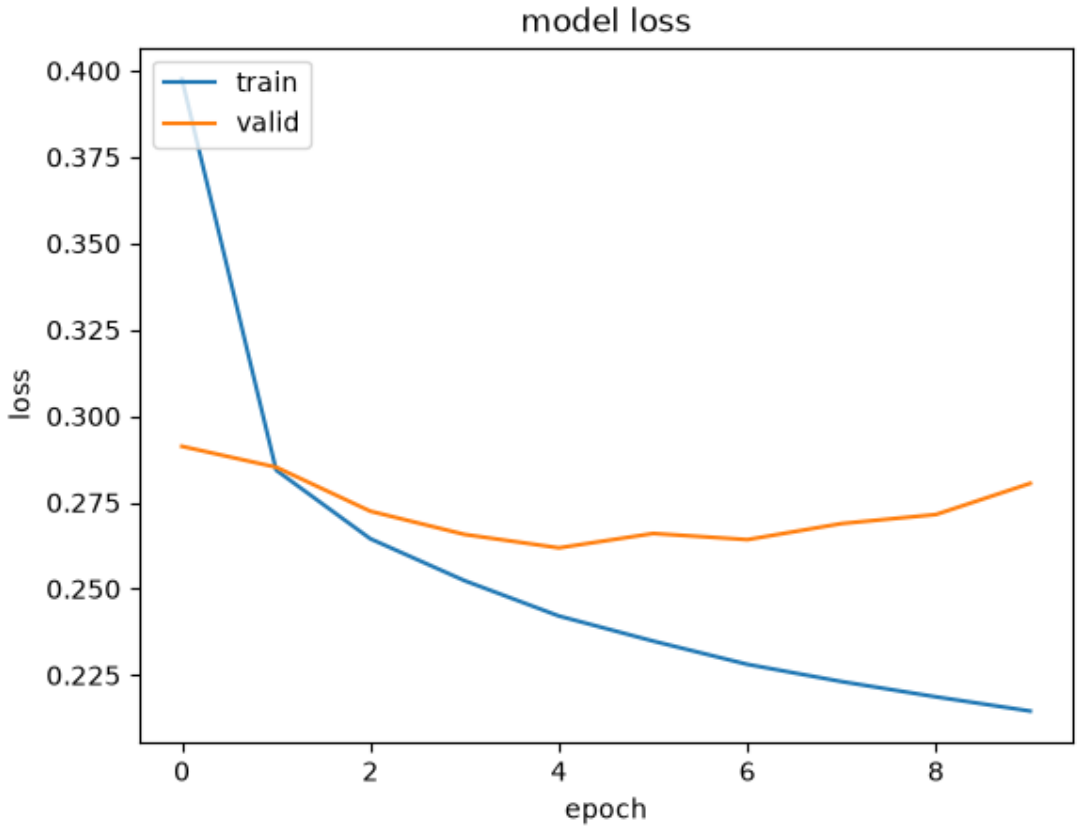
Hình 9: Một số mẫu dự đoán sai của MLP

5.2. Kết quả huấn luyện CNN

Dưới đây là đồ thị độ chính xác và hàm mất mát của mô hình CNN qua 10 epoch:

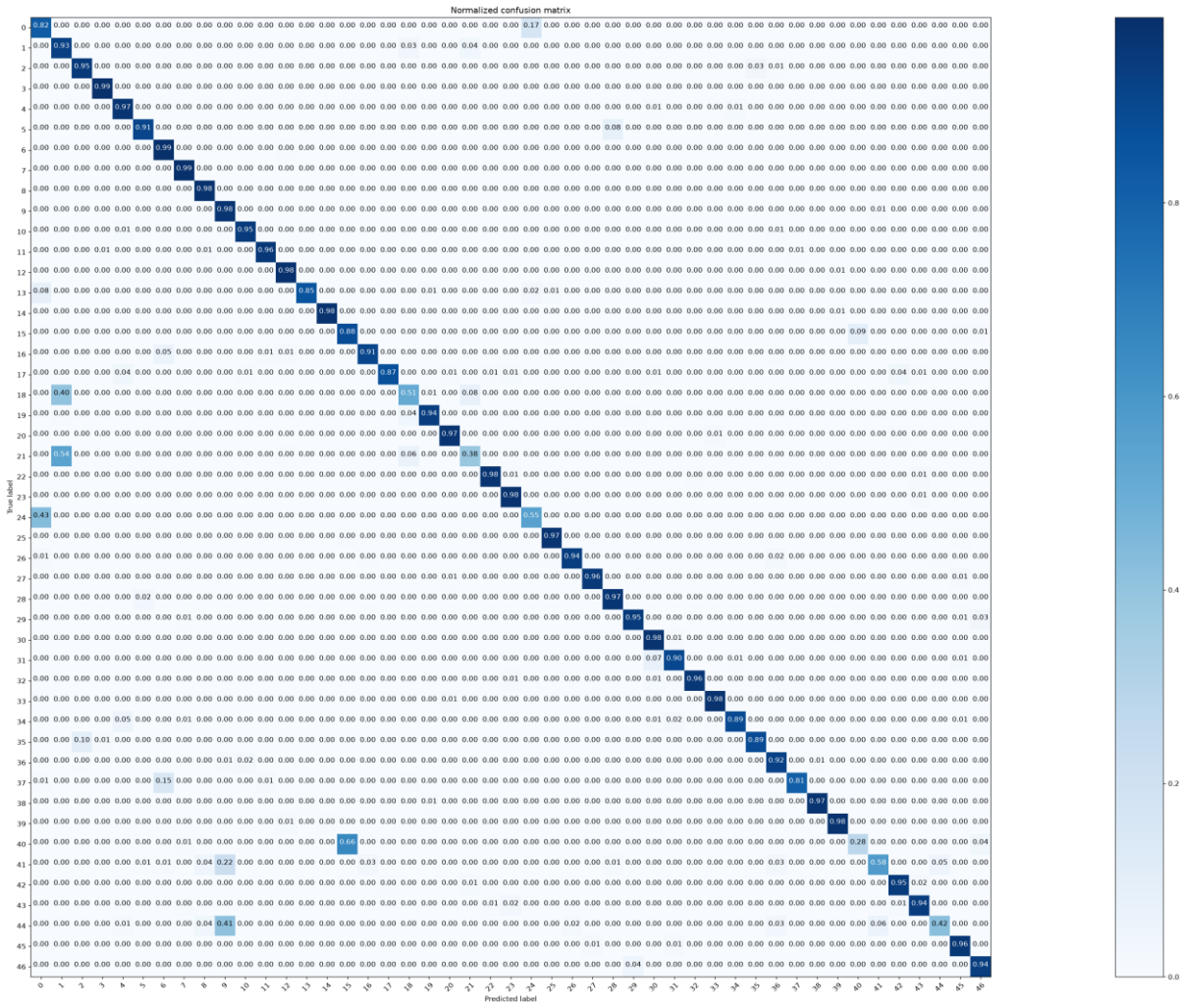


Hình 10: Đồ thị độ chính xác của CNN qua các epoch

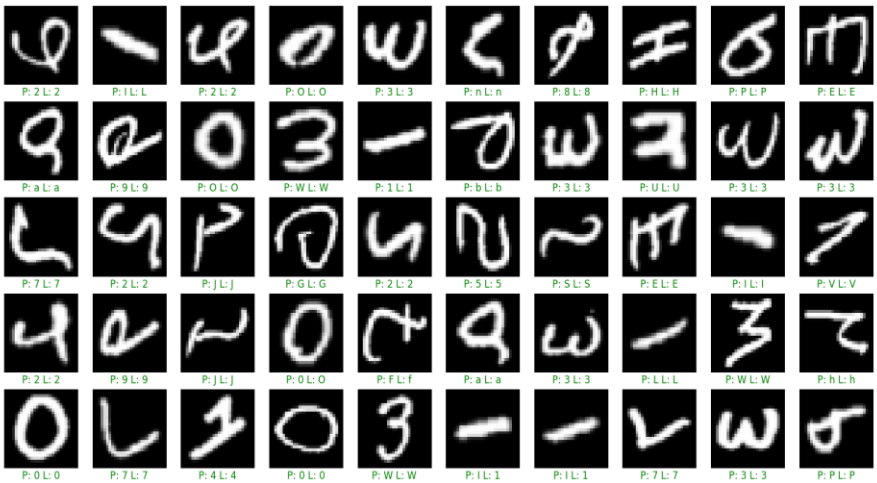


Hình 11: Đồ thị hàm mất mát của CNN qua các epoch

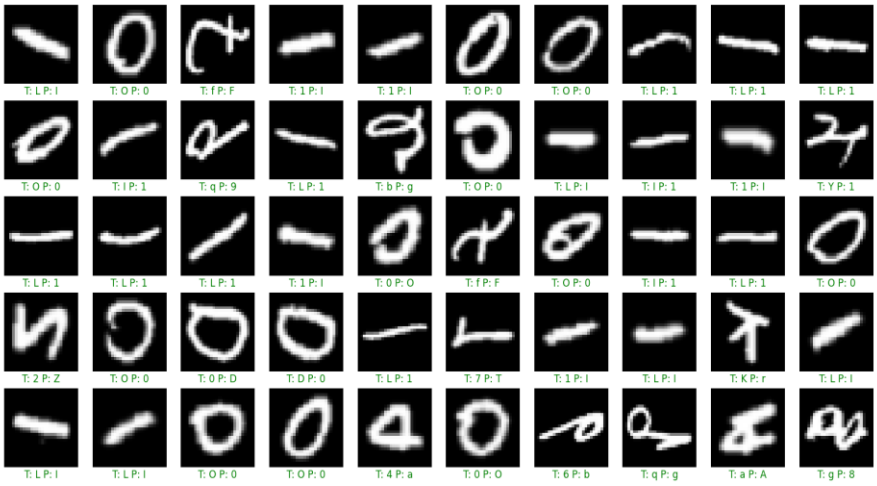
Quan sát: Độ chính xác của CNN tăng nhanh hơn MLP, đạt khoảng 90% sau 10 epoch. Tuy nhiên, khoảng cách giữa accuracy huấn luyện và validation lớn hơn MLP, cho thấy CNN có xu hướng overfitting nhiều hơn.



Hình 12: Ma trận nhầm lẫn của CNN



Hình 13: Một số mẫu dự đoán của CNN



Hình 14: Một số mẫu dự đoán sai của CNN

5.3. So sánh hai mô hình

Tiêu chí	MLP	CNN
Độ chính xác trên tập test	Trên 88%	Khoảng 90%
Tốc độ huấn luyện	Nhanh	Chậm hơn
Số tham số	Ít hơn	Nhiều hơn
Xu hướng overfitting	Ít	Nhiều hơn
Khả năng trích xuất đặc trưng	Còn hạn chế	Mạnh
Kích thước file .keras	~8 MB	~20 MB

Bảng 4: So sánh kết quả giữa MLP và CNN

Nhan xet:

- CNN cho độ chính xác cao hơn MLP nhờ khả năng trích xuất đặc trưng không gian thông qua các lớp tích chập và pooling.
- MLP huấn luyện nhanh hơn vì không cần truyền dữ liệu qua các lớp tích chập.
- CNN có xu hướng overfitting nhiều hơn MLP, cần nhiều kỹ thuật điều chuẩn hơn (dropout, data augmentation,...).
- Cả hai mô hình đều có lỗi tập trung ở các cặp ký tự có hình dạng tương tự."

CHƯƠNG 6: PHÂN TÍCH LỖI

6.1. Phân tích ma trận nhầm lẫn

Từ ma trận nhầm lẫn của cả hai mô hình, có thể nhận thấy:

- Đường chéo chính (các giá trị đúng) chiếm phần lớn, thể hiện mô hình phân loại đúng phần lớn các mẫu.
- Các ô ngoài đường chéo tập trung chủ yếu ở một số cặp lớp cụ thể, cho thấy lỗi có tính hệ thống chứ không phải ngẫu nhiên.
- Một số lớp có độ chính xác thấp hơn đáng kể so với trung bình, đặc biệt là các chữ cái viết thường hoặc các ký tự có hình dạng đặc biệt.

6.2. Các cặp nhầm lẫn phổ biến

"Qua quan sát ma trận nhầm lẫn và các mẫu dự đoán sai, nhóm nhận thấy các cặp ký tự sau đây thường xuyên bị nhầm lẫn:

(1) Chữ O và số 0: Đây là cặp nhầm lẫn phổ biến nhất do hình dạng gần như giống hệt nhau, đặc biệt khi viết tay.
(2) Chữ I, chữ l và số 1: Ba ký tự này có hình dạng rất giống nhau (một đường thẳng đứng), gây khó khăn cho cả MLP và CNN.
(3) Chữ Z và số 2: Khi viết tay, chữ Z và số 2 có thể trông rất giống nhau, đặc biệt với những người viết theo kiểu cách điệu.
(4) Chữ S và số 5: Tương tự, chữ S và số 5 có hình dạng gần giống nhau trong nhiều kiểu viết.
(5) Chữ G và số 6: Cặp này cũng thường xuyên bị nhầm lẫn.
(6) Chữ B và số 8: Hai ký tự này có cấu trúc hình học tương tự.
(7) Chữ D và số 0: Một số kiểu viết chữ D có thể trông giống số 0.

6.3. Nguyên nhân và hướng cải thiện

6.3.1. Nguyên nhân chính

Có ba nguyên nhân chính dẫn đến các lỗi nhận dạng:

(1) Ảnh được lưu trữ ở dạng transpose: Như đã đề cập, ảnh trong EMNIST được lưu theo cách đã xoay 90 độ và lật ngang. Việc không thực hiện np.transpose/np.flip để đưa ảnh về đúng hướng là một trong những nguyên nhân chính gây ra lỗi nhận dạng. Nếu thực hiện thao tác này, các cặp ký tự có hình dạng tương tự sẽ được phân biệt rõ ràng hơn.
(2) Bản chất của bài toán: Một số cặp ký tự vốn có hình dạng rất giống nhau (O/0, I/l/1, S/5,...), ngay cả con người cũng có thể nhầm lẫn khi đọc viết tay. Đây là giới hạn cố hữu của bài toán.
(3) Kiến trúc mô hình chưa tối ưu: Với CNN, nếu các lớp tích chập không được cấu hình tối ưu, mô hình có thể không trích xuất được các đặc trưng phân biệt giữa các cặp ký tự tương tự.

6.3.2. Hướng cải thiện

Dựa trên phân tích, nhóm đề xuất một số hướng cải thiện:

(1) Tiền xử lý ảnh: Thực hiện np.transpose và np.flip để đưa ảnh về đúng hướng hiển thị tự nhiên. Thay đổi này cần được áp dụng đồng bộ trong cell 4 của cả hai notebook.
(2) Tăng cường dữ liệu (Data Augmentation): Áp dụng các phép biến đổi như xoay, dịch chuyển, co giãn để tăng tính đa dạng của dữ liệu huấn luyện, giúp mô hình tổng quát hóa tốt hơn.
(3) Tinh chỉnh siêu tham số: Thử nghiệm với các giá trị learning rate khác nhau, tăng số epoch, điều chỉnh tỷ lệ dropout.
(4) Kiến trúc sâu hơn: Sử dụng các kiến trúc CNN hiện đại hơn như ResNet, VGG để cải thiện khả năng trích xuất đặc trưng.
(5) Sử dụng kỹ thuật Batch Normalization: Giúp ổn định quá trình huấn luyện và tăng tốc độ hội tụ.
(6) Ensemble: Kết hợp nhiều mô hình khác nhau để cải thiện độ chính xác tổng thể.

CHƯƠNG 7: KẾT LUẬN VÀ HẠN CHẾ

7.1. Kết luận

Qua quá trình thực hiện bài tập cuối kỳ, nhóm đã đạt được các mục tiêu đề ra:

(1) Xây dựng thành công hai mô hình MLP và CNN cho bài toán nhận dạng ký tự viết tay trên bộ dữ liệu EMNIST ByMerge với 47 lớp.
(2) Mô hình MLP đạt độ chính xác trên 88% trên tập kiểm thử, mô hình CNN đạt khoảng 90%.
(3) CNN cho kết quả tốt hơn MLP nhờ khả năng trích xuất đặc trưng không gian, nhưng có xu hướng overfitting nhiều hơn và thời gian huấn luyện lâu hơn.
(4) Phân tích lỗi cho thấy các cặp ký tự có hình dạng tương tự (O/0, I/l/1, S/5, Z/2,...) là nguyên nhân chính gây ra lỗi nhận dạng.
(5) Tiền xử lý dữ liệu (đặc biệt là việc xử lý ảnh transpose) đóng vai trò quan trọng trong hiệu năng của mô hình.

Về mặt kiến thức, nhóm đã học được:

- Cách mạng nơ-ron nhận dạng dữ liệu viết tay.
- Quá trình lan truyền ngược (backpropagation) trong mạng nơ-ron.
- Các hàm kích hoạt khác nhau (ReLU, Softmax, LeakyReLU) và ưu điểm của chúng so với Sigmoid.
- Sự khác biệt giữa Gradient Descent thông thường và Stochastic Gradient Descent, cũng như các hàm tối ưu nâng cao (Adam).
- Vai trò của siêu tham số và kỹ thuật điều chuẩn trong huấn luyện mạng nơ-ron.

7.2. Hạn chế

Bên cạnh các kết quả đạt được, bài tập vẫn còn một số hạn chế:

(1) Không xử lý ảnh transpose: Như đã phân tích, việc không thực hiện np.transpose/np.flip là nguyên nhân chính gây ra lỗi nhầm lẫn giữa các cặp ký tự tương tự. Đây là hạn chế lớn nhất của bài tập.
(2) Số epoch hạn chế: Chỉ huấn luyện 10 epoch do hạn chế về thời gian và phần cứng. Với nhiều epoch hơn, mô hình có thể đạt độ chính xác cao hơn.
(3) Chưa tinh chỉnh siêu tham số: Learning rate và các siêu tham số khác được giữ ở giá trị mặc định. Chưa thực hiện grid search hoặc random search để tìm bộ siêu tham số tối ưu.
(4) Chưa áp dụng Data Augmentation: Không sử dụng các kỹ thuật tăng cường dữ liệu, có thể giúp cải thiện khả năng tổng quát hóa của mô hình.
(5) Kiến trúc đơn giản: Cả hai mô hình đều sử dụng kiến trúc tương đối đơn giản. Các kiến trúc hiện đại hơn (ResNet, DenseNet,...) có thể cho kết quả tốt hơn.
(6) Hạn chế về phần cứng: Huấn luyện trên CPU mất nhiều thời gian (hàng chục phút cho mỗi mô hình), hạn chế khả năng thử nghiệm nhiều cấu hình.

7.3. Hướng phát triển

Dựa trên các hạn chế đã chỉ ra, nhóm đề xuất các hướng phát triển trong tương lai:

(1) Áp dụng tiền xử lý ảnh đúng cách (transpose/flip) để cải thiện đáng kể độ chính xác.
(2) Thử nghiệm với các kiến trúc CNN sâu hơn (ResNet, VGG, EfficientNet,...).
(3) Áp dụng Data Augmentation để tăng tính đa dạng của dữ liệu huấn luyện.
(4) Sử dụng kỹ thuật Transfer Learning từ các mô hình đã được huấn luyện trước trên ImageNet.
(5) Xây dựng ứng dụng demo cho phép người dùng vẽ ký tự và nhận dạng trực tiếp.
(6) Mở rộng bài toán sang nhận dạng chuỗi ký tự (text recognition) thay vì từng ký tự riêng lẻ.

TÀI LIỆU THAM KHẢO

[1] G. Cohen, S. Afshar, J. Tapson, A. van Schaik, EMNIST: an extension of MNIST to handwritten letters, 2017. [Online]. Available: https://arxiv.org/pdf/1702.05373.pdf

[2] StatQuest with Josh Starmer, Gradient Descent. [Online]. Available: https://www.youtube.com/watch?v=sDv4f4s2SB8

[3] StatQuest with Josh Starmer, Stochastic Gradient Descent. [Online]. Available: https://www.youtube.com/watch?v=vMh0zPT0tLI

[4] V. Bushaev, Adam - Latest Trends in Deep Learning Optimization. [Online]. Available: https://towardsdatascience.com/adam-latest-trends-in-deep-learning-optimization-6be9a291375c

[5] 3Blue1Brown, Neural Networks. [Online]. Available: https://www.youtube.com/playlist?list=PLZHQObOWTQDNU6R1_67000Dx_ZCJB-3pi

[6] Machine Learning Co ban, Multi-layer Perceptron and Backpropagation. [Online]. Available: https://machinelearningcoban.com/2017/02/24/mlp/

[7] M. Nielsen, Neural Networks and Deep Learning. [Online]. Available: http://neuralnetworksanddeeplearning.com

[8] T. Kumar, Digit Recognition CNN tutorial. [Online]. Available: https://www.kaggle.com/tarunkr/digit-recognition-tutorial-cnn-99-67-accuracy

[12] Keras, Layer weight initializer. [Online]. Available: https://keras.io/api/layers/initializers/

[13] Keras, Adam Optimizer. [Online]. Available: https://keras.io/api/optimizers/adam/

[14] S. Ruder, Overview of gradient descent optimization algorithms. [Online]. Available: https://ruder.io/optimizing-gradient-descent/

[17] Wikipedia, Rectifier Neural Network. [Online]. Available: https://en.wikipedia.org/wiki/Rectifier_(neural_networks)

[19] Keras, Categorical Crossentropy Loss. [Online]. Available: https://keras.io/api/losses/probabilistic_losses/

[24] mlfromscratch, Optimizers Explained. [Online]. Available: https://mlfromscratch.com/optimizers-explained/

[25] J. Wu, Convolutional Neural Networks. [Online]. Available: https://cs.nju.edu.cn/wujx/teaching/15_CNN.pdf

[26] V. Sharma, S. Rai, A. Dev, A Comprehensive Study of Artificial Neural Networks. [Online]. Available: http://citeseerx.ist.psu.edu/viewdoc/download?doi=10.1.1.468.9353

[27] Stanford, CS231n: Convolutional Neural Networks for Visual Recognition. [Online]. Available: https://cs231n.github.io/convolutional-networks/

[28] PyImageSearch, Keras Conv2D and Convolutional Layers. [Online]. Available: https://www.pyimagesearch.com/2018/12/31/keras-conv2d-and-convolutional-layers/

[29] ResearchGate, Hyperparameter Optimization and Regularization on Fashion-MNIST Classification. [Online]. Available: https://www.researchgate.net/publication/334947180

[30] Wikipedia, Learning rate. [Online]. Available: https://en.wikipedia.org/wiki/Learning_rate